

s4.06 — E-commerce recommendations from browsing history

Index

- [Interview framing](#)
- [Requirements — IC3 baseline](#)
- [Capacity and consequence — IC3 baseline](#)
- [Architecture and flow — IC3 baseline](#)
- [API and data model — IC3 baseline](#)
- [Event ingestion and feature freshness — IC3 baseline](#)
- [Candidate generation, ranking, and serving versions — IC3 baseline](#)
- [Failure, operations, and security](#)
- [Alternatives and IC4 stretch](#)
- [Level signal — IC3 baseline vs IC4 stretch](#)
- [Interview pushback and validation](#)
- [Recall card](#)
- [References](#)

Interview framing

- Target: OCI IC3 ownership of a bounded platform component.
 - `IC3 baseline` is what must land in every phase.
 - `IC4 stretch` adds architecture changes under stronger constraints.
 - These are preparation labels, not Oracle's official rubric.
- Evidence: first-hand 1p3a report retained in [the bank](#).
 - The report gives the prompt, not the interviewer's accepted answer.
 - The architecture below is a proposed interview answer.
 - It does not claim to reproduce Oracle's implementation.
 - Model quality is out of scope for the round; the platform that makes any model safe to serve is in scope.
- OCI framing for a business prompt
 - Control plane: model registry, rollouts, serving pointer, privacy deletions.
 - Data plane: the recommendation query and the behavior-event ingest.
 - Static stability: serving nodes keep the last-good model, index, and serving pointer locally, so the data plane survives a control-plane outage.
 - Tenancy: a tenant is a storefront or merchant; events, features, indexes, and models are tenant-scoped.
 - Blast radius: one bad model version or one tenant's event storm must not degrade every surface.
- Why this is hard
 - Naive: one table of views per user, a nightly job that computes "people who viewed X also viewed Y", a SQL query at page load.
 - First break: latency and freshness.
 - Scoring `10M` items per request does not fit a `150 ms` page budget.
 - A nightly job cannot see the item the user viewed thirty seconds ago.
 - Second break: the feedback loop.
 - The model learns from what it showed, so logging the wrong thing (every generated candidate, or nothing) corrupts the labels.
 - A feature schema change or a bad model reaches every user at once unless versions are pinned and rolled out.
 - Third break: privacy and eligibility.

- A good score does not make an item purchasable, in stock, or allowed in the user's region.
- A deletion request must reach the event log, the features, and the impressions, not just the profile row.
- Minimum primitives
 - An immutable, deduplicated behavior-event log partitioned by subject.
 - An online feature store with a freshness watermark and a schema version.
 - Two-stage serving: bounded candidate retrieval, then ranking, then eligibility filtering.
 - A serving pointer that pins model, feature schema, and index generation together.
 - Impression logging of what was shown, joined to later outcomes for training.
- 🗣️ "Recommendations are a feedback loop with a latency budget: log what you showed, pin what you served, and never let a score authorize an item."
- Opening script
 - "I will treat this as an event-ingestion pipeline feeding an online feature store, with a two-stage retrieval-and-rank serving path and a versioned control plane for models."
 - "First I confirm traffic, latency budget, freshness expectation, the surfaces, and the privacy obligations."
 - "Then I show the numbers: about 100k events per second and 20k recommendation requests per second, which force a bounded candidate set and a per-request cache rather than full scoring."
 - "The deep dives are event deduplication and feature freshness, and candidate generation plus ranking under a pinned serving version with impression logging."
 - "I will close with cold start, the failure table with fallbacks, rollout and rollback of models, and deletion across derived stores."

Phase	Clock	Deliverable
Intro and behavioral slice	00:00–05:00	90-second pitch; confirm the prompt
Requirements	05:00–11:00	Scope, clarifying answers, primary invariant
Estimates and contract	11:00–17:00	Event and request rates, feature and index sizes; API semantics
Architecture	17:00–30:00	Diagram, ingest versus serve versus control plane, one request trace
Deep dive	30:00–45:00	Dedup and watermark; retrieval, ranking, serving pointer, impressions
Operations and tradeoffs	45:00–50:00	Failure table, fallback, rollout, privacy, wrap script
Wrap-up	50:00–54:00	Four-sentence recap and weakest assumption
Questions for interviewer	54:00–60:00	Two role-specific questions

- Clock note
 - If the interviewer states 45 minutes, drop the intro to 2 minutes and compress each phase proportionally.
 - The interviewer's clock wins over this table.

Requirements — IC3 baseline

- Numerical targets are interview assumptions unless an adjacent primary source states otherwise.

Question	Assumed answer	Why it changes the design
Users and catalog?	50M monthly active users, 10M catalog items	Full scoring is impossible per request; forces retrieval then rank
Traffic?	100k behavior events/s peak, 20k recommendation requests/s peak	Ingest is a log problem; serving is a cache and budget problem
Latency budget?	Recommendation p99 < 150 ms inside page render	Feature fetch, retrieval, filter, and rank must share about 100 ms
Freshness?	Session behavior reflected within 60 s; models retrained daily	Online feature updates, batch training; no online training on the request path
Surfaces?	Home page, product page "similar items", cart	Different candidate sources per surface; one serving pointer per surface

Question	Assumed answer	Why it changes the design
Privacy?	Pseudonymous subject token; deletion within 30 d; opt-out honored at serve	Deletion must traverse event log, features, impressions; opt-out is a serve-time check
Multi-tenant?	Yes, tenant is a storefront	Per-tenant events, features, index, and model; no cross-tenant training by default

- Functional scope
 - Accept batched behavior events (view, add-to-cart, purchase) with deduplication.
 - Maintain online per-subject features from recent behavior.
 - Serve a ranked list of items for a surface with a reason code.
 - Log impressions of what was actually shown.
 - Register, roll out, and roll back model versions with health gates.
 - Delete a subject's data across derived stores.

Requirement	Target	Enforced by
Recommendation latency	p99 < 150 ms	Bounded candidates (≤ 500), per-request budget, response cache 60 s per (subject, surface)
Recommendation availability	99.99% including fallback	Popular-by-context fallback served from local cache when features or index fail
Event ingest latency	p99 < 50 ms to 202	Append to partitioned log after dedup check; no synchronous feature computation
Feature freshness	Session features within 60 s of the event	Stream processor lag alarm on the event-time watermark
Model rollout safety	Canary 5% cohort; auto-rollback on conversion drop > 2%	Rollout controller with health gates and previous version retained
Impression logging coverage	$\geq 99\%$ of served responses	Impression write on the serving path with bounded buffer and replay
Deletion completion	Within 30 d	Work request tracked across event log, feature KV, impression store
Candidate cap	500 per request, limit ≤ 50	Server-capped regardless of client request

- Invariants
 - Every accepted event has a unique event_id; a replay never double-counts.
 - A request is served by one pinned triple: model version, feature schema version, index generation.
 - The response logs the version used, even when a fallback served the result.
 - Impressions record displayed items, not every generated candidate.
 - A candidate must pass eligibility (availability, region, tenant rules, opt-out) after scoring; a score never authorizes an item.
 - Training uses only events before the training cutoff and before each label's event time.
- Non-goals
 - Model architecture selection and offline metric tuning.
 - Online learning on the request path.
 - Cross-tenant transfer learning in v1.
 - Real-time bidding or ad ranking.

Capacity and consequence — IC3 baseline

- Behavior events
 - 100k/s peak, assume 30k/s average, about 2.6B events per day.
 - At about 300 B each that is 30 MB/s peak and about 780 GB per day raw.
 - 90 d raw retention is on the order of 70 TB in object storage.
 - Decision: append-only partitioned log, raw copies in object storage, never a relational insert per event.

- Partition by `subject_token` so one subject's events are ordered.
- Dedup store
 - 2.6B events per day × a 7 d retry window is about 18B keys.
 - At roughly 64 B per key that is on the order of 1.2 TB.
 - Decision: dedup by `event_id` with a TTL equal to the supported retry window, sharded by `event_id` hash.
 - Beyond the window, compaction dedups again by `event_id` before features are computed.
- Online features
 - 50M subjects × about 2 KB of session and profile features is about 100 GB.
 - Decision: a key-value store keyed by `(tenant, subject_token, feature_schema_version)` with an idle TTL.
 - Inactive subjects are evicted and rebuilt from the log on demand.
- Item index
 - 10M items × 128 float32 dimensions is about 5 GB of embeddings.
 - An HNSW graph roughly doubles that; it fits in memory on one node per replica.
 - Decision: immutable index generations built offline and swapped atomically.
 - Never mutate the live graph on the serving path.
- Serving
 - 20k/s requests × 500 candidates is 10M candidate scores per second.
 - A lightweight ranker at about 50 μs per candidate on a batch is 500 s of CPU per second, so about 600 cores at 80% utilization, an order of magnitude estimate.
 - Decision: two-stage retrieval then rank, a 60 s response cache per `(subject, surface)`, and precomputed lists for inactive subjects.
 - Full catalog scoring at 10M × 20k/s is 2×10^{11} scores per second and is not an option.
- Impressions
 - 20k/s requests × 20 shown items is 400k impression rows per second.
 - Decision: log impressions as one record per request with an items array into the event log.
 - The companion's relational impression table is the logical model; the physical store is columnar and partitioned by day.
- Training
 - Daily batch over 90 d of events with labels joined from impressions.
 - Decision: batch training with a training cutoff; the serving path never trains.
- So what
 - Ingest is a log and dedup problem.
 - Serving is a budget problem solved by bounding candidates and caching.
 - Say both in the first two minutes.

Architecture and flow — IC3 baseline

```
[ingest / data plane]
Client -> POST /behavior-events -> validator (authn, tenant, batch cap, dedup by event_id)
      -> event log partitioned by subject_token [async]
      -> stream processor -> online feature KV (session features, watermark)
      -> object storage (raw, immutable, 90 d)

[serve / data plane / hot path]
Client -> API gateway -> recommendation service
      -> response cache (subject, surface, 60 s)
      -> online feature KV (subject features, event watermark)
      -> candidate generation: ANN index (item embeddings) + co-visitation index + session recency
      -> eligibility filter (availability, region, tenant rules, opt-out, already purchased)
      -> ranker (model pinned by serving pointer) -> top-K with reasonCode
      -> impression record (request_id, shown items, versions) -> event log [async]
      fallback: popular-by-context list from local cache when features or index are unavailable

[control plane]
Batch trainer (daily) -> model registry (immutable artifact, feature schema version, training cutoff)
Index builder -> immutable item index generation
Rollout controller -> serving pointer (tenant, surface, cohort) -> canary -> health gates -> promote or rollback
Privacy workflow -> deletion work request across event log, feature KV, impressions
```

- Validator owns event acceptance.
 - Mechanism bought: authenticated subject, bounded batch, dedup by `event_id` before append, 202 with accepted ids.
 - Cost paid: a dedup-store lookup on every event; the store must scale with the retry window.
- Event log owns the source of truth for behavior.
 - Mechanism bought: append-only, partitioned by subject, replayable into features and training ([s3.06](#)).
 - Cost paid: at-least-once delivery downstream; every consumer must be idempotent by `event_id`.
- Stream processor owns online features.
 - Mechanism bought: session features (last 20 items, category counts with decay) updated within 60 s, with an event-time watermark.
 - Cost paid: late and out-of-order events; features must be order-insensitive or sorted by event time.
- Online feature KV owns serve-time reads.
 - Mechanism bought: one read per request at about 5 ms for features plus watermark.
 - Cost paid: a second copy of state that can lag the log; staleness must be visible to the ranker.
- Candidate generator owns recall.
 - Mechanism bought: ANN over item embeddings plus a co-visitation index bounds the set to ≤ 500 in about 20 ms.
 - Cost paid: recall is limited to what the index can reach; new items need a content embedding or an exploration slot.
- Eligibility filter owns the authorization boundary.
 - Mechanism bought: availability, region, tenant rules, and opt-out applied after scoring from a cached catalog snapshot.
 - Cost paid: catalog snapshot staleness; an out-of-stock item can appear for up to the snapshot TTL.
- Ranker owns ordering.
 - Mechanism bought: a pinned model version scores bounded candidates with subject, item, and context features.
 - Cost paid: the largest slice of the latency budget; model size is constrained by it.
- Model registry and rollout controller own the control plane.
 - Mechanism bought: immutable artifacts, a serving pointer per (tenant, surface, cohort), canary and rollback ([s3.03](#)).

- Cost paid: every version change is a rollout with health gates, so shipping a model is slower than copying a file.
 - Impression logger owns the feedback loop.
 - Mechanism bought: records what was shown with versions so later clicks and purchases attribute correctly.
 - Cost paid: 400k rows per second of write volume that must not block the response.
- Client sends `POST /recommendation-queries {surface, limit, context}` with the subject from auth.
- Gateway authenticates, resolves the tenant, and caps `limit` at 50.
- Service checks the response cache for `(tenant, subject, surface)`; a hit under 60 s returns immediately.
- Service reads the serving pointer for `(tenant, surface, cohort)`: model version, feature schema version, index generation.
- Service reads subject features and the event watermark from the feature KV under the pinned schema version.
- Candidate generator queries the pinned index generation with the subject embedding and recent items; caps at 500.
- Eligibility filter drops unavailable, restricted, opted-out, and already-purchased items using the cached catalog snapshot.
- Ranker scores the survivors with the pinned model and returns the top `limit` with a `reasonCode` per item.
- Service writes an impression record `{requestId, items, positions, versions, fallback flag}` to the event log asynchronously.
- Service returns 200 `{requestId, modelVersion, items}`; the client later posts `recommendation-impressions` for what it actually rendered.

API and data model — IC3 baseline

Surface	Contract	Idempotency	Conflict/error behavior
<code>POST /behavior-events</code>	<code>{events:[{eventId, itemId, eventType, occurredAt, impressionId?}]}; returns 202 {acceptedIds}</code>	Per-event <code>eventId</code> ; duplicates are accepted and not re-appended	400 for malformed events; batch capped at 500; 429 on per-subject or per-tenant rate
<code>POST /recommendation-queries</code>	<code>{surface, limit, context}; returns 200 {requestId, modelVersion, items:[{itemId, reasonCode}]}</code>	Not idempotent; each call is a new request id	<code>limit</code> capped server-side; fallback list returned with <code>fallback: true</code> when dependencies fail
<code>POST /recommendation-impressions</code>	<code>{requestId, items:[{itemId, position}]}; returns 202</code>	Idempotent per <code>(requestId, itemId, position)</code>	Unknown <code>requestId</code> returns 404; late impressions past 24 h are dropped and counted
<code>POST /recommendation-models</code>	<code>{artifactRef, featureSchemaVersion, trainingCutoff}; returns 201 model</code>	Idempotency-Key; immutable registry entry	Same key with different payload returns 409; operator-only
<code>POST /recommendation-rollouts</code>	<code>{modelVersion, cohort, healthGates}; returns 202 operation</code>	Idempotency-Key; one active rollout per <code>(tenant, surface)</code>	409 if a rollout is already in flight for the surface
<code>GET /recommendation-rollouts/{id}</code>	<code>{state, metrics, activeVersions}</code>	Read	Operator view; 404 for another tenant's id
<code>POST /recommendation-profile-deletions</code>	<code>{subjectRef, reason}; returns 202 operation</code>	Idempotency-Key; one open deletion per subject	Authorized privacy workflow; progress tracked as a work request (s3.01)

- Path note
 - Paths follow the API companion; `/v1` prefixes are implied.
 - Rollouts and deletions return 202 with an operation id because they span stores and time.

State	Key	Fields/invariant
Event log record	(tenant, subject_token) partition; event_id	event_type, item_id, event_time, receive_time, impression_id, schema_version; append-only
Dedup store	(tenant, event_id)	payload_hash, accepted_offset; TTL equals the supported retry and replay window
Online feature KV	(tenant, subject_token, feature_schema_version)	Features, event_watermark, updated_at; idle TTL
Item candidate index	(tenant, index_generation, item_id)	Embedding, category, eligibility_version; immutable per generation
Serving pointer	(tenant, surface, cohort)	model_version, feature_schema_version, index_generation; one compatible triple
Model registry	(tenant_id, model_id, model_version)	artifact_ref, feature_schema_version, training_cutoff, state; immutable once registered
Impression	(tenant_id, impression_id); UNIQUE (tenant_id, request_id, item_id, position)	subject_token, model_id, model_version, at; joins later outcomes
Catalog snapshot	(tenant, item_id)	Availability, region rules, price band; refreshed every 60 s; cached on serving nodes

```

CREATE TABLE recommendation_models (
  tenant_id text NOT NULL, model_id text NOT NULL, model_version bigint NOT NULL,
  artifact_ref text NOT NULL, feature_schema_version text NOT NULL,
  training_cutoff timestamptz NOT NULL, state text NOT NULL,
  PRIMARY KEY (tenant_id, model_id, model_version)
);
CREATE TABLE recommendation_impressions (
  tenant_id text NOT NULL, impression_id text NOT NULL, request_id text NOT NULL,
  subject_token text NOT NULL, item_id text NOT NULL, position integer NOT NULL,
  model_id text NOT NULL, model_version bigint NOT NULL, at timestamptz NOT NULL,
  PRIMARY KEY (tenant_id, impression_id),
  UNIQUE (tenant_id, request_id, item_id, position),
  FOREIGN KEY (tenant_id, model_id, model_version) REFERENCES recommendation_models
);
CREATE INDEX recommendation_attribution ON recommendation_impressions
(tenant_id, subject_token, at, impression_id);

```

- Schema notes
 - The SQL is the logical model from the companion.
 - At 400k impressions per second the physical store is the event log plus daily columnar partitions; the relational table is used for operator queries over a sample or a short window.
 - Model, feature schema, and index generation form one compatible serving configuration; a pointer update replaces the whole triple.
 - Retention is separate per store: raw events 90 d, impressions 180 d, features until idle TTL.
 - A deletion needs lineage through every derived store, which is why every derived record carries subject_token.

Event ingestion and feature freshness — IC3 baseline

- Acceptance
 - Authenticate the subject from the token; the client never supplies a subject id.
 - Validate eventType, itemId, and occurredAt (reject future timestamps beyond clock skew of 5 min).
 - Check (tenant, event_id) in the dedup store; on hit, return the id as accepted without re-appending.
 - Append to the event log partition for subject_token; record accepted_offset in the dedup store.
 - Return 202 {acceptedIds}.

- Consistency boundary
 - The dedup write and the log append are not one transaction.
 - Order: append first, then record the dedup entry; a crash between them leaves a possible duplicate in the log.
 - Downstream consumers dedup again by `event_id` inside the retry window, so the log may contain duplicates and the features may not.
 - Say the residual out loud: exactly-once is enforced at the consumer, not at the wire.
- Ordering and time
 - Partitioning by `subject_token` gives per-subject order at the log.
 - `event_time` (`occurredAt`) drives features; `receive_time` drives operations.
 - The stream processor keeps a per-partition event-time watermark; the feature KV stores it as `event_watermark`.
 - Late events past the watermark by more than `10 min` are applied to features but excluded from the current session window.
- Session features
 - Last `20` viewed item ids with timestamps.
 - Category and brand counts with exponential decay (half-life `7 d`).
 - Add-to-cart and purchase sets for the last `30 d` to suppress already-bought items.
 - Order-insensitive aggregates so out-of-order application converges.
- Feature schema version
 - Every feature write carries `feature_schema_version`.
 - A model declares the schema version it was trained on; the serving pointer only binds compatible versions.
 - Schema change is a rollout: dual-write the new version, backfill from the log, then move pointers.
 - Defeater: an unversioned feature change silently feeds incompatible input to a model and the metric drop looks like a model problem.
- Batch versus online
 - Online: feature updates within `60 s`.
 - Batch: daily training with a `training_cutoff`; labels come from impressions joined to later events.
 - Offline evaluation uses only events before the cutoff and, per label, before the label's event time, so future information does not leak.
 - Online training on every request couples serving latency to training failures and is not needed for freshness.

```

INGEST(batch):
  for e in batch (cap 500):
    validate(e); subject = auth.subject
    if dedup.contains(tenant, e.eventId): accepted.add(e.eventId); continue
    offset = log.append(partition(tenant, subject), e)      # at-least-once
    dedup.put(tenant, e.eventId, hash(e), offset, ttl = retry_window)
    accepted.add(e.eventId)
  return 202 accepted

FEATURES(partition stream):
  for e in stream ordered by offset:
    if seen(e.eventId) within window: continue           # consumer-side dedup
    f = kv.get(tenant, subject, schema_version)
    apply(f, e)                                           # order-insensitive aggregates
    f.event_watermark = max(f.event_watermark, e.event_time)
    kv.put(f)

```

- Exact failure
 - Client retries a batch after a lost `202`: the dedup store returns the ids as accepted; features count each view once.
 - Dedup store unavailable: accept with at-least-once and let the consumer dedup; alarm because the residual window is now the consumer's memory, not the store's TTL.

- Stream processor lag of 10 min: the watermark exposes it; the ranker down-weights session features older than 5 min and the metric dashboard labels the cohort as stale.

Candidate generation, ranking, and serving versions — IC3 baseline

- Latency budget inside p99 150 ms (assumed split)
 - Gateway and cache check 5 ms.
 - Feature and pointer reads 10 ms.
 - Candidate generation 20 ms.
 - Eligibility filter 10 ms.
 - Ranking 30 ms.
 - Impression enqueue and response 5 ms.
 - Headroom for tail 70 ms.
- Candidate generation
 - Subject embedding from session features (average of recent item embeddings with recency weights) queried against the HNSW index for the pinned generation.
 - Co-visitation index for the product page: items viewed in the same sessions as the current item, precomputed daily.
 - Session recency: the last 20 items' nearest neighbors merged in.
 - Union, dedupe, cap at 500.
 - Surfaces choose sources: home uses embedding plus popular; product page uses co-visitation plus embedding; cart uses complements.
- Eligibility after scoring
 - Availability and region from the cached catalog snapshot (60 s TTL).
 - Tenant rules (excluded categories, age gates).
 - Opt-out flag on the subject.
 - Already-purchased set from features.
 - A candidate that fails eligibility is dropped even with the top score; a score never authorizes an item.
- Ranking
 - The pinned model scores each candidate with subject features, item features, and context (surface, device, time).
 - Return top limit with a reasonCode (viewed_similar, bought_together, popular_in_category, fallback).
 - Add one exploration slot per 20 positions for new items so they can earn signal.
- Serving pointer and versions
 - Read once per request; cached locally with a 30 s refresh and last-good fallback ([s3.03](#)).
 - The triple is replaced atomically; a request never mixes a new model with an old index.
 - The response and the impression record carry the versions used.
 - If the fallback served, log fallback as the version and set fallback: true so metrics separate the cohorts.
- Impressions and labels
 - Log what was shown with positions, not every generated candidate.
 - The client confirms rendered items via recommendation-impressions; the server-side record is the backstop.
 - Labels join impressions to later clicks and purchases by impression_id within an attribution window of 7 d.
 - Defeater: logging generated candidates as impressions produces negatives the user never saw and biases the next model.
- Cold start
 - New subject: popular-by-context (surface, category, region) until 3 events exist, then session-based retrieval.
 - New item: content embedding from attributes and text so it is reachable by ANN, plus the exploration slot.
 - Fallback traffic is labeled so cold-start quality is measured separately.

- Precomputation for inactive subjects
 - Subjects idle for more than 7 d receive a nightly precomputed list keyed by (tenant, subject, surface).
 - Serving reads the list and skips retrieval, which cuts CPU for the long tail.

```

SERVE(tenant, subject, surface, limit, ctx):
    limit = min(limit, 50)
    if r = cache.get(tenant, subject, surface): return r
    p = pointer.get(tenant, surface, cohort(subject))          # {model, schema, index}, last-good on
miss                                                         # may be missing -> cold start
    f = features.get(tenant, subject, p.schema)
    if f is None or index.unavailable(p.index):
        items = popular.get(tenant, surface, ctx.region)[:limit]; version = "fallback"
    else:
        cand = union(ann(p.index, embed(f)), covisit(ctx.item), recent_neighbors(f))[:500]
        cand = [c for c in cand if eligible(c, catalog.snapshot(), f, tenant.rules)]
        items = rank(p.model, f, cand, ctx)[:limit]; version = p.model
    req = new_request_id()
    impressions.enqueue(req, items, version, fallback = version == "fallback") # never blocks
    cache.put(tenant, subject, surface, items, ttl = 60 s)
    return {requestId: req, modelVersion: version, items}

```

- Exact failure
 - Index generation swap during a request: the request holds a reference to the old generation until it completes; the old generation is released after the last reader exits.
 - Ranker timeout at 30 ms: return candidates ordered by retrieval score with reasonCode = retrieval_only and count it; never return an empty page.
 - Pointer store unavailable: serve the last-good cached triple indefinitely and alarm; this is the static-stability path.

Failure, operations, and security

Failure	Blast radius	Detection	Degraded behavior	Recovery
Online feature KV unavailable	Personalization for all subjects in the region	KV error rate, fallback ratio	Serve popular-by-context with fallback: true; response cache keeps recent lists	Restore KV; rebuild hot subjects from the event log
ANN index node loss	Retrieval capacity in one cell	Retrieval error rate, p99	Remaining replicas serve; shed to co-visitation and popular lists above capacity	Reload the immutable generation on a new node
Model registry or pointer store down (control plane / metadata store)	Rollouts and pointer changes	Control-plane health, pointer refresh failures	Serving continues on last-good pointer and cached artifacts (static stability)	Restore store; pointers reconcile on next refresh
Bad model version rolled out	Canary cohort (5%)	Conversion or CTR drop beyond gate	Rollout controller flips the pointer back to the previous version	Mark version FAILED; keep artifact for post-mortem
Event log consumer lag	Feature freshness for lagging partitions	Watermark age alarm at 5 min	Ranker down-weights stale session features; cohort labeled stale	Scale consumers; replay from offset
Dedup store unavailable	Duplicate counting inside the retry window	Dedup error rate	Accept at-least-once; consumers dedup within their memory window	Restore store; backfill from log offsets
Catalog snapshot stale	Out-of-stock or restricted items shown	Snapshot age above 60 s	Serve with last snapshot; suppress items flagged unavailable at render by the client	Refresh feed; alarm on age

Failure	Blast radius	Detection	Degraded behavior	Recovery
Subject event storm (bot at 1k/s)	One partition, one subject's features	Per-subject rate metric	Per-subject rate limit 429; features cap window size	Block token; features unaffected beyond cap
Deletion workflow partial failure	One subject's derived data	Work request stuck past 24 h	Serving treats the subject as opted out until deletion completes	Retry per store; lineage lists what remains

- Observe
 - Recommendation p99 and fallback ratio per surface and tenant.
 - Feature watermark age per partition.
 - Dedup hit ratio and duplicate rate downstream.
 - Impression coverage (responses with a logged impression).
 - Canary metrics per model version: CTR, conversion, latency.
 - Deletion work-request age.
- Retry discipline
 - Event clients retry batches with the same `event_ids` and exponential backoff with jitter; the server dedups.
 - Serving retries a feature read once within budget, then falls back; never retry past the latency budget.
 - Impression writes use a bounded local buffer and replay from the serving log; drops are counted, not retried inline.
 - Rollout and deletion operations are work requests with server-side retries and a terminal state.
- Tenant isolation
 - Events, features, index generations, and models are keyed by tenant; a tenant's model never scores another tenant's items.
 - Per-tenant ingest and query quotas; a tenant's event storm cannot lag another tenant's partitions when partitions are tenant-prefixed.
 - Operator ids from another tenant return 404, not 403, to avoid existence leaks.
 - Cross-tenant training is off by default and requires an explicit agreement and a separate model lineage.
- Rollout and rollback safety
 - Model rollout: register artifact, canary 5% cohort, health gates for 1 h, promote, keep the previous version hot for instant rollback.
 - Feature schema rollout: dual-write, backfill from the log, shadow-score, then move pointers; contract the old schema after no pointer references it.
 - Index generation rollout: build, validate recall against the previous generation on a sample, swap the pointer, release the old generation after readers drain.
 - Serving fleet deploys are canaried and must survive with the local last-good pointer during the deploy.
- Privacy and security
 - Subject tokens are pseudonymous; the mapping to account ids lives in a separate authorized store.
 - Features exclude sensitive categories by policy; coarse region only.
 - Opt-out is a serve-time check and a training exclusion.
 - Deletion is a work request that traverses event log tombstones, feature KV delete, impression redaction, and training exclusion lists; retrained models drop the subject at the next cycle.
 - Deletion timing: the 30 d target is an assumption; the applicable regulation sets the actual bound.

Alternatives and IC4 stretch

Option	What it is	Strength	Weakness	Choose when
Popularity by context	Top items per surface, category, region	Zero personalization cost; always available	No lift for users with history	Fallback and A/B control in every design
Item-to-item co-visitation	"Viewed X also viewed Y" precomputed	Simple, explainable, strong on product pages	Cannot generalize to new items; sparse for long tail	Product page baseline

Option	What it is	Strength	Weakness	Choose when
Matrix factorization	Latent user and item factors from interactions	Well understood; cheap to serve	Retrain to include new users and items	Batch-only personalization
Two-tower embeddings plus ANN	Learned subject and item embeddings, nearest-neighbor retrieval	Scales retrieval to 10M items; handles content features	Needs an index pipeline and version discipline	Baseline retrieval stage
Sequence or session model	Model over the ordered recent events	Best freshness and intent capture	Heavier serving; needs ordered features	Real-time personalization requirement
Online learning on requests	Update weights on each event	Immediate adaptation	Couples latency to training; unreproducible versions	Almost never on the serving path

- IC4 stretch — real-time personalization within seconds
 - Move session features to the serving node with a per-subject in-memory window fed by the event stream.
 - Replace the average-embedding subject vector with a session model scored on the last 20 events.
 - The rest of the design holds: pinned versions, impressions, and eligibility do not change.
- IC4 stretch — 10× load (200k requests/s, 1M events/s)
 - Precompute lists for every subject idle more than 1 d, so live retrieval serves only active sessions.
 - Shard the index by tenant and quantize embeddings (int8) to cut memory 4×.
 - Raise the response-cache TTL per surface where freshness allows; home tolerates 5 min.
 - Partition the event log by (tenant, subject) hash with more partitions; consumers scale linearly.
- IC4 stretch — cost and blast radius
 - Cell the serving path per region and tenant tier so a bad index or model reaches one cell.
 - Training and index builds are the dominant compute cost; run them per tenant tier, not per tenant, below a size threshold.
 - Raw event retention in object storage is the dominant storage cost; 90 d is the knob.
- IC4 stretch — migration and rollout
 - Shadow-score a new model on live traffic without serving it; compare rankings offline before the canary.
 - Interleaving experiments split positions between two models on one page to cut A/B duration.
- Weakest assumption
 - Browsing history is predictive enough to beat popularity for most sessions.
 - If most users have fewer than 3 events, the fallback serves the majority and the personalization stack earns little.
 - Mitigation: measure the fallback ratio and the A/B lift first; keep popularity as a first-class tier, not an error path.
- Over-engineering warning
 - Do not put online training on the request path.
 - Do not build a real-time feature store for a home page that tolerates 5 min staleness.
 - Do not log every generated candidate as an impression to “get more data”; it corrupts the labels.

Level signal — IC3 baseline vs IC4 stretch

Phase	IC3 must land	IC4 adds	Separating signal
Requirements	Traffic, latency budget, freshness, surfaces, privacy obligations	Names that model quality is out of scope and eligibility is the authorization boundary	Separates platform correctness from model accuracy
Estimates	100k/s events, 20k/s queries, 500 candidates, feature and index sizes	Shows full scoring is 2×10^{11} per second and impressions are 400k/s, so the store choice follows	A number forces two-stage serving and the event log

Phase	IC3 must land	IC4 adds	Separating signal
Architecture	Ingest, serve, control-plane split with one trace and a fallback path	Static stability on last-good pointer and artifacts; per-tenant cells	Every component has a mechanism and a price
Deep dive	Dedup by <code>event_id</code> , watermark, pinned version triple, impressions of what was shown	Consumer-side exactly-once residual; leakage-free training cutoff; exploration slot	Reasons about the feedback loop, not just the request
Operations	Failure table with fallback, canary and rollback, deletion workflow	Schema dual-write rollout, shadow scoring, cost knobs	Volunteers rollout and privacy before being asked
Wrap	Recap and batch-versus-online tradeoff	Names the predictive-power assumption first	Identifies the flaw before the interviewer does

Interview pushback and validation

- Q1: Why not train the model online on every request?
 - **IC3 baseline**
 - Freshness comes from online feature updates within `60 s`, not from online training.
 - Online training couples serving latency to training failures and makes versions unreproducible.
 - **IC4 stretch**
 - If sub-minute adaptation is required, use a session model over the recent event window with fixed weights; the weights still ship through the registry and canary.
- Q2: The top-scored item is out of stock. What happens?
 - **IC3 baseline**
 - Eligibility runs after scoring against a cached catalog snapshot; a score never authorizes an item.
 - The snapshot TTL of `60 s` bounds the window.
 - **IC4 stretch**
 - The client applies a last-second availability check at render and reports suppressed positions so labels stay honest.
- Q3: A brand-new user lands on the home page.
 - **IC3 baseline**
 - Popular-by-context until `3` events exist, then session-based retrieval.
 - The response carries `reasonCode = popular_in_category` and the impression is labeled fallback.
 - **IC4 stretch**
 - Measure cold-start quality as its own cohort and use contextual signals (referrer, device, region) before any history exists.
- Q4: A new model version drops conversion by `5%` an hour after rollout.
 - **IC3 baseline**
 - The canary cohort is `5%`; the health gate at `2%` flips the pointer back to the previous version automatically.
 - The previous version stays hot, so rollback is a pointer change.
 - **IC4 stretch**
 - Shadow-score and interleave before the canary; check the feature schema version first because a schema mismatch looks like a bad model.
- Q5: Someone adds a feature and redeploys the stream processor. What breaks?
 - **IC3 baseline**
 - Nothing if the feature schema version changed and the serving pointer only binds compatible versions.
 - Everything silently if the version did not change; the model reads shifted input.
 - **IC4 stretch**
 - Enforce the version at write time and at pointer update time, and dual-write during the migration so rollback is instant.

- Q6: A user asks for deletion. What does it touch?
 - **IC3 baseline**
 - A work request tombstones the event log, deletes feature KV entries, redacts impressions, and adds the subject to the training exclusion list.
 - Serving treats the subject as opted out until the request completes.
 - **IC4 stretch**
 - Lineage through derived stores is the hard part; every derived record carries `subject_token` so deletion is enumerable, and retrained models drop the subject at the next cycle.
- Q7: Why not log every generated candidate as an impression?
 - **IC3 baseline**
 - The user never saw them, so they become false negatives and bias the next model toward what was already shown.
 - Log displayed items with positions; the client confirms rendered items.
 - **IC4 stretch**
 - Keep a sampled log of the candidate set separately for offline recall analysis, clearly labeled as not impressions.
- Q8 (premise reversal): Why personalize at all? A popularity list per category might get most of the lift with a fraction of the machinery.
 - **IC3 baseline**
 - Popularity is the fallback and the A/B control in this design, so it ships first and is always available.
 - Personalization earns its cost only when the A/B shows lift on the fallback ratio the numbers predict.
 - **IC4 stretch**
 - Build the platform pieces that popularity also needs, the event log, impressions, versions, and rollout, and add retrieval and ranking as a second phase gated by measured lift.
- Validation scenarios
 - Client retries an event batch after a lost `202`; features count each view once.
 - Two events for one subject arrive out of order; features converge.
 - Feature KV outage; fallback ratio rises and latency stays inside budget.
 - Index generation swap under load; no request mixes generations.
 - Bad model canary; automatic rollback within the gate window.
 - Feature schema change without a version bump; the pointer refuses to bind.
 - Out-of-stock item at the top score; the filter drops it.
 - Deletion request; every derived store reports completion.
 - Bot subject at `1k` events/s; other subjects unaffected.
 - Tenant A operator on tenant B rollout id returns `404`.

Recall card

- "Log what you showed, pin what you served, never let a score authorize an item."
- "Ingest is a deduplicated, subject-partitioned log; features are a derived view with a watermark."
- "Retrieve `500`, filter for eligibility, rank, return `50`; full scoring is not an option."
- "Model, feature schema, and index generation move as one triple behind a canary."
- "Popularity is the fallback and the control, not an error path."
- 5 numbers: `100k/s` events
 - `20k/s` queries
 - `500` candidates, `50` returned
 - `60 s` feature freshness
 - `5%` canary
- Failures to rehearse
 - Feature KV outage with fallback.

- Bad model canary and rollback.
- Unversioned feature change.
- 3 defeaters: online training on the request path
 - Logging candidates as impressions
 - Trusting a score as eligibility
- Done when: the serving pseudocode is written cold
 - The version triple and the dedup residual are explained in 30 s each
- Wrap-up script
 - “The design is a deduplicated event log feeding an online feature store, a two-stage retrieve-filter-rank serving path with a per-request cache, and a control plane that pins model, feature schema, and index generation behind a canary.”
 - “The key invariant is that every response logs the version it served and the impressions it showed, and eligibility is applied after scoring so a score never authorizes an item.”
 - “The main tradeoff is batch training with online features: freshness within a minute and reproducible versions, in exchange for a daily model lag and a feature store to operate.”
 - “My weakest assumption is that browsing history beats popularity for most sessions; I would measure the fallback ratio and A/B lift first and keep popularity as a first-class tier.”

References

- [Ranked question and report provenance.](#)
- [OCI role framing.](#)
- [Detailed API and schema companion.](#)
- Sibling designs
 - [s3.06 durable event log](#): partitioned append-only log and consumer offsets.
 - [s3.03 versioned configuration distribution](#): serving-pointer refresh with last-good fallback.
 - [s3.01 async control-plane resource API](#): work requests for rollouts and deletions.
 - [s1.03 thread-safe LRU cache](#): response cache and catalog snapshot cache.
 - [s1.05 internal metrics ingestion](#): high-rate ingest analogy.
- Reference check: 2026-09-10.
 - Public papers and documentation support the mechanisms cited next to each link.
 - Capacity values, latency splits, TTLs, and the canary gate remain proposed interview assumptions.
- [Covington, Adams, Sargin, “Deep Neural Networks for YouTube Recommendations” \(RecSys 2016\)](#): two-stage candidate generation then ranking.
- [Malkov and Yashunin, “Efficient and robust approximate nearest neighbor search using HNSW” \(2018\)](#): the ANN index used for retrieval.
- [Google, “Rules of Machine Learning”](#): training-serving skew and logging what was shown.
- [Apache Kafka design](#): partitioned logs, ordering per partition, at-least-once delivery.
- [GDPR Article 17](#): right to erasure; the 30 d target above is an assumption, the regulation says without undue delay.
- [PostgreSQL constraints](#): keys and uniqueness in the companion schema.